Analytics Vidhya

A Guide to LLM and RAG Evaluation

Framework	Supported Metrics	Input/Output	Integration	Official Docs
Ragas	Context Precision/ Recall, Entity Recall, Response Relevancy, Faithfulness, Noise Sensitivity, Semantic Similarity, etc.	**Input**: Samples with reference , retrieved_contexts (or reference_contexts). **Output**: scores (0–1) for each metric.	Python SDK, integrate with LangChain or custom eval loops.	Ragas Docs (metrics: ContextRecall, ContextPrecision, etc.)
DeepEval (Confident AI)	**RAG**: Answer Relevancy, Faithfulness, Contextual Relevancy/ Precision/Recall; **Agents**: TaskCompletion, ToolCorrectness, PlanQuality, etc.	**Input**: LLMTestCase or LLMTrace ; can include expected_output . **Output**: JSON with score and reason.	Python CLI/SDK, integrates with LangChain (CallbackHandler), or custom code.	DeepEval Docs
Opik (Comet.ml)	Heuristic: BLEU/ ROUGE/ BERTScore, Regex, JSON validity, etc.; LLM-Judge: Answer Relevance, Context Precision/ Recall, Hallucination, Task Completion, Trajectory Accuracy, etc.	**Input**: traces or logs of inputs/outputs. **Output**: dashboard metrics and scores.	Python/ TypeScript SDK, integrates with Hugging Face Datasets, LangChain, wandb.	Opik Docs
LangSmith	Custom evaluators; supports accuracy, F1, BLEU, as well as any model-based or prompt-based metric.	**Input**: LangChain traces or any JSON of inputs/ **outputs**. Output: experiment records with metrics.	Native in LangChain (evaluate() , tracing). Logs data to W&B backed UI.	LangSmith Docs
Vertex AI Evaluation	Custom (pointwise scores 0–N, pairwise comparisons), plus built-in metrics (ROUGE, BLEU). Uses proprietary judge models for pairwise/ pointwise.	Input: Dataset in BigQuery or Cloud Storage; candidate outputs + references. Output: scores, charts.	Google Cloud service; integrates with Vertex pipelines and UI.	Vertex AI GenAI Eval

Modern AI systems often augment large language models (LLMs) with retrieval over external data (“RAG”) to improve accuracy and factuality.

Evaluating such systems requires a structured approach across multiple layers: the retrieval pipeline (data ingestion, chunking, embeddings, search) and the generation pipeline (prompting, response).

We must measure retrieval quality (precision/recall of context) and generation quality (answer relevance, faithfulness). Agentic flows add further metrics (task completion, tool-use correctness, plan fidelity). Leading frameworks (Ragas, DeepEval, Opik, LangSmith, Vertex AI, etc.) offer pre-built metrics and pipelines for automated evaluation.

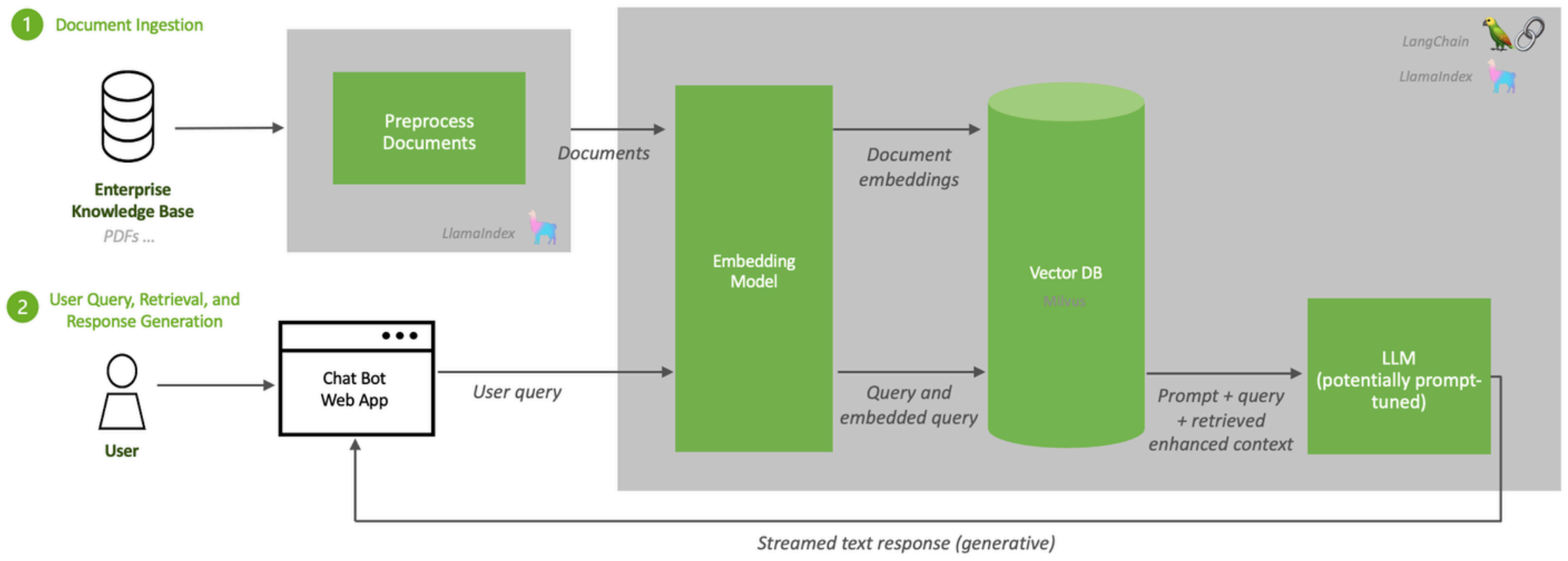
This post provides a deep-dive with definitions, best practices, and comparisons. It includes slide-ready diagrams (see RAG pipeline below), evaluation examples, and tables of tools/metrics. Example thresholds (e.g. context recall ≥0.85, precision ≥0.7 for Q&A) and CI workflows are also discussed. All information is drawn from official documentation (OpenAI, Google, Anthropic, LangChain, etc.) to ensure accuracy.

RAG Architecture & Pipeline

A Retrieval-Augmented Generation (RAG) system typically has two phases :

- Offline ingestion of documents
- Online retrieval + LLM generation when a query arrives.

The NVIDIA pipeline (Figure below) illustrates these components.



Document Ingestion: Raw data (PDFs, web, DBs) are loaded and split (“chunked”) into meaningful units (passages). Chunking strategies (fixed-size, overlapping, semantic clustering, LLM-based) significantly impact recall. ChromaDB guidelines emphasise designing collection schema, chunk size, and metadata for the target search modalities

Embedding & Storage: Each chunk is converted to a vector via an embedding model, then stored in a vector database (Chroma, Milvus, etc.) . Embedding quality (model choice) and index structure (ANN parameters) affect retrieval.

Retrieval: At runtime, the user query is also embedded. A vector search (dense, lexical, or hybrid) retrieves the top-k chunks. Optionally, a re-ranking step (using another model) can reorder results. This yields a context set.

Prompt Construction: The retrieved context is combined with the query into a final prompt. Prompt engineering may include system instructions or templates.

LLM Generation: The prompt is fed to the LLM (e.g. GPT-4o, Claude 3 Sonnet, Gemini) to generate an answer . This completes the pipeline.

Retrieval Metrics (Context Quality)

Once the system retrieves context, we evaluate how good that context is. Common metrics compare the retrieved chunks to ground-truth context (if known) or references. Key metrics include:

Context Precision

Fraction of retrieved content that is relevant. Formally, the proportion of retrieved tokens or chunks that match the ground truth . (High precision means few irrelevant chunks.)

Context Recall

Fraction of relevant content that was retrieved. It measures not missing any key info . (High recall ensures we found most needed facts.)

Context Entity Recall

Fraction of ground-truth entities (names, dates, etc.) covered by the retrieval . Useful for fact-based queries (e.g. QA).

Noise Sensitivity

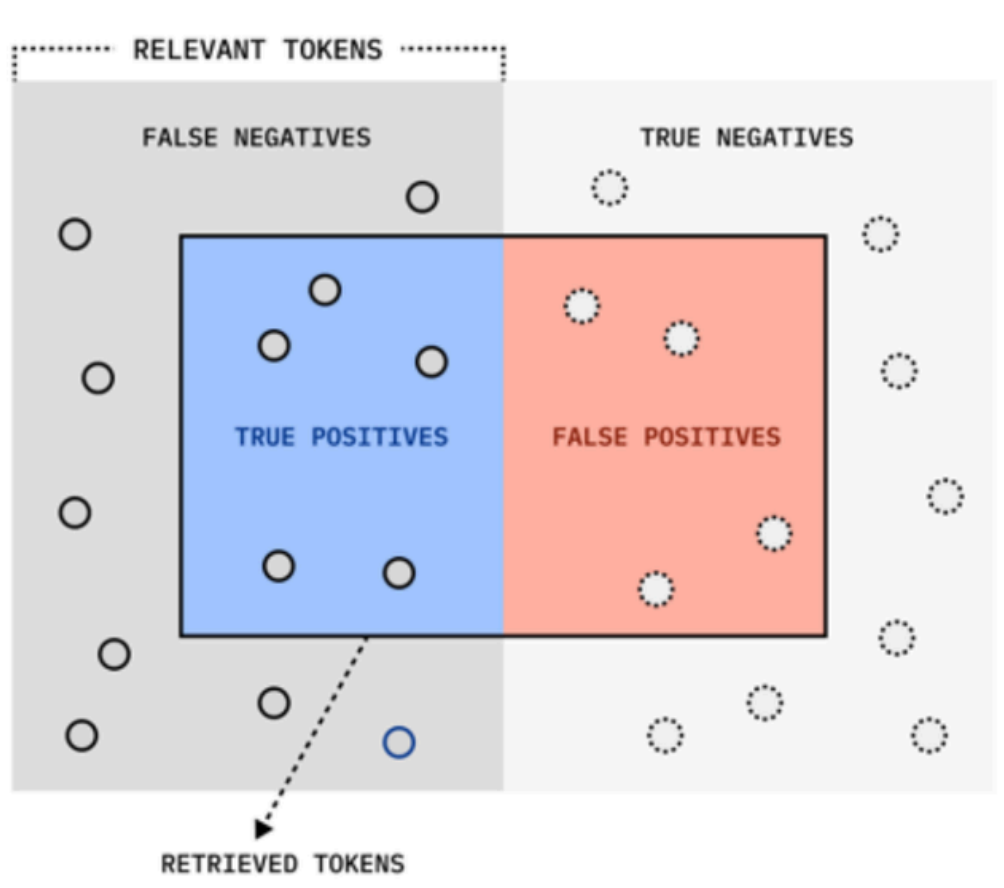
Fraction of retrieved content that is off-topic or hallucinated (higher noise lowers precision). Ragas defines this to penalize irrelevant context

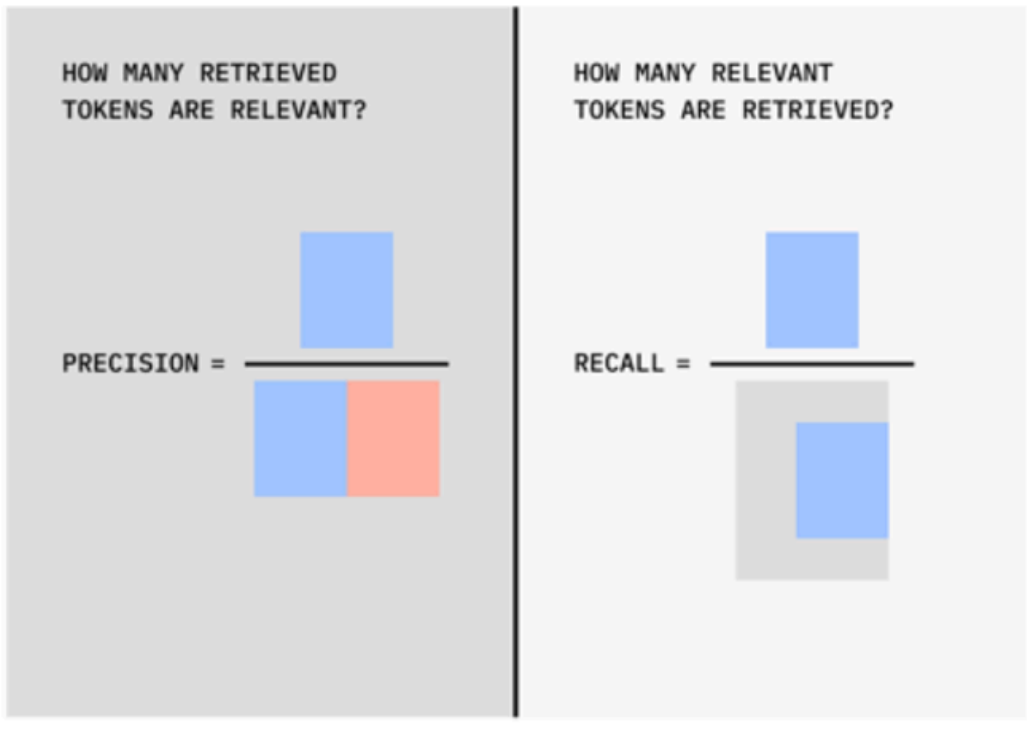
Overlap/IOU

For token-level relevance, some evaluations use precision/recall at token or sentence overlap levels (e.g. Intersection-over-Union of relevant tokens).

Retrieval Ranking Metrics

When ordering matters, we use IR metrics like Mean Reciprocal Rank (MRR) and nDCG (Normalized Discounted Cumulative Gain). These assess whether relevant chunks appear near the top of the ranking.





Precision vs Recall concept (true/false positives) – useful for interpreting context precision/recall

Generation Metrics (Answer Quality)

After retrieval, we assess the LLM’s answer against the query and reference answer. Common metrics include:

Answer Relevancy

Does the answer address the question? DeepEval and Opik define Answer Relevance to check if the reply stays on-topic.

This is often a binary or graded score.

Faithfulness

Is the answer factually consistent with provided context? Faithfulness metrics flag hallucinations or unsupported claims. DeepEval’s Faithfulness metric scores this by comparing answer to context , and Opik has a Hallucination judge metric.

Semantic Similarity

Vector-similarity (e.g. cosine of sentence embeddings) between the generated answer and a reference answer.

Helps judge content overlap beyond exact words.

NLP Scores

BLEU, ROUGE, METEOR, BERTScore, etc., against ground-truth answers. These capture n-gram overlap (ROUGE) or semantic similarity (BERTScore). They are heuristic but widely used. (Opik includes BLEU/ROUGE metrics).

Non-Task Metrics

In dialogue or agent contexts, measures like Usefulness, Conversational Coherence, and safety (bias/toxicity) may be included

For open-ended generation, often human evaluation or LLM-as-judge is needed. For classification or scoring tasks, accuracy or F1 can apply

Agentic / Multi-Agent Metrics

When LLMs act as agents (calling tools, planning, multi-step tasks), we measure higher-level behaviors:

Task Completion

Did the agent accomplish the overall goal? (DeepEval's TaskCompletion and Opik's Agent Task Completion judge if final objectives are met).

Tool Correctness

If the agent uses tools/APIs, did it use them appropriately? (DeepEval's Tool Correctness and Opik's Agent Tool Correctness evaluate this).

Plan Quality and Adherence

Did the agent create a sound plan and follow it? DeepEval defines Plan Quality/Adherence, and Opik has Trajectory Accuracy (how well steps match an expected path).

Step Efficiency

Were tool calls minimal? (DeepEval's Step Efficiency rewards concise solutions).

Argument Correctness

For multi-step reasoning, metrics can score the correctness of each intermediate inference.

Multi-turn/Chat Metrics

For conversational agents, metrics like Knowledge Retention, Role Adherence, and dialogue-level coherence are used (DeepEval lists these for chatbots).

In practice, agentic evaluation often involves specialized test suites (multi-step tasks) where the agent's full trace is scored against multiple criteria (Anthropic uses evaluation harnesses, Opik logs agent trajectories).

LLM-as-a-Judge Protocols

Instead of hard metrics, a second LLM can judge outputs.

Key protocols:

Evaluation Prompting

Define a clear instruction for the judge LLM (e.g. “Rate the answer 1–5 based on correctness”). DeepEval’s G-Eval lets you craft natural-language criteria for grading correctness or style . Opik’s LLM-as-Judge metrics use fixed prompts under the hood.

Pointwise vs Pairwise

Pointwise asks the LLM to score one answer against criteria. Pairwise asks to compare two answers (e.g. “Which answer is better?”). Google recommends pairwise for stable model comparisons . Both approaches are supported: DeepEval and Vertex AI can do both.

Calibration & Consistency

LLM judges can be stochastic. Use techniques like chain-of-thought or multiple samples to improve consistency. Some frameworks (DeepEval) use tools to decompose scoring for robustness.

LLM Choice

Often a strong model (e.g. GPT-5n or Claude Sonnet) is used as the judge. Opik by default uses GPT-5-nano . LangSmith lets you pick models for evaluators.

Evaluation Workflows & AgentOps

Evaluations should be systematic and continuous.

Here are some key practices:

Design Test Suites: Define an eval objective and build a dataset of inputs. Combine real user queries with handcrafted “golden” examples (with reference answers) that cover normal cases, edge cases, and adversarial cases . For RAG, include queries with complex reasoning, multi-hop, synonyms, and noise. Tavily’s Real-Time Dataset Generator demonstrates automating dataset creation from web sources .

Scoped Tests: Write unit tests for components (e.g. retrieval-only tests, generation-only tests) and end-to-end tests. Use pointwise eval (score each case) or pairwise (compare systems). Incorporate human judgments for subjective aspects.

Continuous Integration: Automate evals in CI pipelines. LangSmith’s **evaluate()** (Python SDK) and Vertex AI’s GenAI Evaluation API support running batches of tests programmatically. Whenever models or code change, rerun the eval suite and compare metrics to baselines.

Best practice is “eval-driven development”: evaluate early and often, calibrate automated scores with
human labels , and iteratively refine.

Tools & Frameworks Comparison

Framework	Supported Metrics	Input/Output	Integration	Official Docs
Ragas	Context Precision/ Recall, Entity Recall, Response Relevancy, Faithfulness, Noise Sensitivity, Semantic Similarity, etc.	**Input**: Samples with reference , retrieved_contexts (or reference_contexts). **Output**: scores (0–1) for each metric.	Python SDK, integrate with LangChain or custom eval loops.	Ragas Docs (metrics: ContextRecall, ContextPrecision, etc.)
DeepEval (Confident AI)	**RAG**: Answer Relevancy, Faithfulness, Contextual Relevancy/ Precision/Recall; **Agents**: TaskCompletion, ToolCorrectness, PlanQuality, etc.	**Input**: LLMTestCase or LLMTrace ; can include expected_output . **Output**: JSON with score and reason.	Python CLI/SDK, integrates with LangChain (CallbackHandler), or custom code.	DeepEval Docs
Opik (Comet.ml)	Heuristic: BLEU/ ROUGE/ BERTScore, Regex, JSON validity, etc.; LLM-Judge: Answer Relevance, Context Precision/ Recall, Hallucination, Task Completion, Trajectory Accuracy, etc.	**Input**: traces or logs of inputs/outputs. **Output**: dashboard metrics and scores.	Python/ TypeScript SDK, integrates with Hugging Face Datasets, LangChain, wandb.	Opik Docs
LangSmith	Custom evaluators; supports accuracy, F1, BLEU, as well as any model-based or prompt-based metric.	**Input**: LangChain traces or any JSON of inputs/ **outputs**. Output: experiment records with metrics.	Native in LangChain (evaluate(), tracing).	LangSmith Docs
Vertex AI Evaluation	Custom (pointwise scores 0–N, pairwise comparisons), plus built-in metrics (ROUGE, BLEU). Uses proprietary judge models for pairwise/ pointwise.	Input: Dataset in BigQuery or Cloud Storage; candidate outputs + references. Output: scores, charts.	Google Cloud service; integrates with Vertex pipelines and UI.	Vertex AI GenAI Eval